Welcome to Colab!

Google Colab is available in VS Code!

Try the new Google Colab extension for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the ***Extensions*** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the ***Select Kernel*** button in the top right.
- Click ***Colab*** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our announcement blog here.

🎁 Free Pro Plan for Gemini & Colab for US College Students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research, and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free offer at gemini.google/students.
Get the Colab free offer at colab.research.google.com/signup.
Terms Apply.

Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the getting started with google colab ai.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to Google AI Studio and log in with your Google account.
- Create an API key.
- Use a quickstart for Python, or call the REST API using curl.

Discover Gemini's advanced capabilities

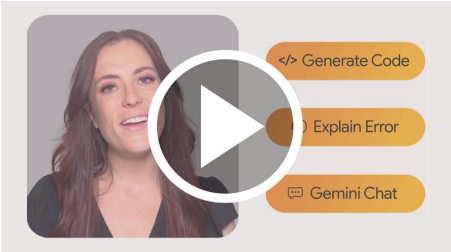
- Play with Gemini multimodal outputs, mixing text and images in an iterative way.
- Discover the multimodal Live API (demo here).
- Learn how to analyze images and detect items in your pictures using Gemini (bonus, there's a 3D version as well!).
- Unlock the power of Gemini thinking model, capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use Gemini grounding capabilities to create a report on a company based on what the model can find on internet.
- Extract invoices and form data from PDF in a structured way.
- Create illustrations based on a whole book using Gemini large context window and Imagen.

To learn more, check out the Gemini cookbook or visit the Gemini API documentation.

Colab now has AI features powered by Gemini. The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

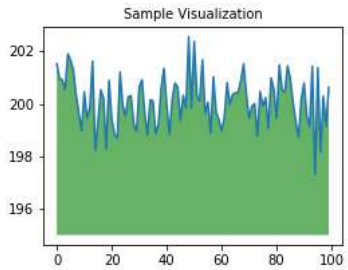
You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under Working with Data.

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"""![{alt}]({image})"""))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including GPUs and TPUs, regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like RAPIDS cuDF that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10 minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the machine learning examples below.

More Resources

Working with Notebooks in Colab

- Overview of Colab
- Guide to Markdown
- Importing libraries and installing dependencies
- Saving and loading notebooks in GitHub
- Interactive forms
- Interactive widgets

Working with Data

- Loading data: Drive, Sheets, and Google Cloud Storage
- Charts: visualizing data
- Getting started with BigQuery

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the full course website for more.

- Intro to Pandas DataFrame
- Intro to RAPIDS cuDF to accelerate pandas
- Getting Started with cuML's accelerator mode

Using Accelerated Hardware

- Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API
- Train a Vision Transformer (ViT) for image classification with JAX
- Text classification with a transformer language model using JAX

Featured examples

- Train a miniGPT language model with JAX AI Stack
- LoRA/QLoRA finetuning for LLM using Tunix
- Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA
- Loading Hugging Face Transformers Checkpoints
- 8-bit Integer Quantization in Keras
- Float8 training and inference with a simple Transformer model
- Pretraining a Transformer from scratch with KerasHub
- Simple MNIST convnet
- Image classification from scratch using Keras 3
- Image Classification with KerasHub

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.head())
```

```
   age  sex  chest pain type  resting bp s  cholesterol  fasting blood sugar  \
0   40    1                2           140          289                    0
1   49    0                3           160          180                    0
2   37    1                2           130          283                    0
3   48    0                4           138          214                    0
4   54    1                3           150          195                    0

   resting ecg  max heart rate  exercise angina  oldpeak  ST slope  target
0            0             172                0      0.0         1       0
1            0             156                0      1.0         2       1
2            1              98                0      0.0         1       0
3            0             108                1      1.5         2       1
4            0             122                0      0.0         1       0
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.shape)
```

```
(1190, 12)
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1190 entries, 0 to 1189
Data columns (total 12 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   age                  1190 non-null   int64
```

```
 1   sex                  1190 non-null   int64
 2   chest pain type      1190 non-null   int64
 3   resting bp s         1190 non-null   int64
 4   cholesterol          1190 non-null   int64
 5   fasting blood sugar  1190 non-null   int64
 6   resting ecg          1190 non-null   int64
 7   max heart rate       1190 non-null   int64
 8   exercise angina      1190 non-null   int64
 9   oldpeak              1190 non-null   float64
 10  ST slope             1190 non-null   int64
 11  target               1190 non-null   int64
dtypes: float64(1), int64(11)
memory usage: 111.7 KB
None
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.describe())
```

```
               age          sex  chest pain type  resting bp s  cholesterol  \
count  1190.000000  1190.000000      1190.000000   1190.000000  1190.000000
mean     53.720168     0.763866         3.232773    132.153782   210.363866
std       9.358203     0.424884         0.935480     18.368823   101.420489
min      28.000000     0.000000         1.000000      0.000000     0.000000
25%      47.000000     1.000000         3.000000    120.000000   188.000000
50%      54.000000     1.000000         4.000000    130.000000   229.000000
75%      60.000000     1.000000         4.000000    140.000000   269.750000
max      77.000000     1.000000         4.000000    200.000000   603.000000

       fasting blood sugar  resting ecg  max heart rate  exercise angina  \
count          1190.000000  1190.000000     1190.000000      1190.000000
mean              0.213445     0.698319      139.732773         0.387395
std               0.409912     0.870359       25.517636         0.487360
min               0.000000     0.000000       60.000000         0.000000
25%               0.000000     0.000000      121.000000         0.000000
50%               0.000000     0.000000      140.500000         0.000000
75%               0.000000     2.000000      160.000000         1.000000
max               1.000000     2.000000      202.000000         1.000000

           oldpeak     ST slope       target
count  1190.000000  1190.000000  1190.000000
mean      0.922773     1.624370     0.528571
std       1.086337     0.610459     0.499393
min      -2.600000     0.000000     0.000000
25%       0.000000     1.000000     0.000000
50%       0.600000     2.000000     1.000000
75%       1.600000     2.000000     1.000000
max       6.200000     3.000000     1.000000
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.columns)
```

```
Index(['age', 'sex', 'chest pain type', 'resting bp s', 'cholesterol',
       'fasting blood sugar', 'resting ecg', 'max heart rate',
       'exercise angina', 'oldpeak', 'ST slope', 'target'],
      dtype='object')
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.age)
```

```
0       40
1       49
2       37
3       48
4       54
        ..
1185    45
1186    68
1187    57
1188    57
1189    38
Name: age, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.sex)
```

```
0       1
1       0
2       1
3       0
4       1
       ..
1185    1
1186    1
1187    1
1188    0
1189    1
Name: sex, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df ['chest pain type'])
```

```
0       2
1       3
2       2
3       4
4       3
       ..
1185    1
1186    4
1187    4
1188    2
1189    3
Name: chest pain type, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['resting bp s'])
```

```
0       140
1       160
2       130
3       138
4       150
       ...
1185    110
1186    144
1187    130
1188    130
1189    138
Name: resting bp s, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df.cholesterol)
```

```
0       289
1       180
2       283
3       214
4       195
       ...
1185    264
1186    193
1187    131
1188    236
1189    175
Name: cholesterol, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['fasting blood sugar'])
```

```
0       0
1       0
2       0
3       0
4       0
       ..
1185    0
1186    1
1187    0
1188    0
```

```
1189    0
Name: fasting blood sugar, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['resting ecg'])
```

```
0       0
1       0
2       1
3       0
4       0
       ..
1185    0
1186    0
1187    0
1188    2
1189    0
Name: resting ecg, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['max heart rate'])
```

```
0       172
1       156
2        98
3       108
4       122
       ...
1185    132
1186    141
1187    115
1188    174
1189    173
Name: max heart rate, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['exercise angina'])
```

```
0       0
1       0
2       0
3       1
4       0
       ..
1185    0
1186    0
1187    1
1188    0
1189    0
Name: exercise angina, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['oldpeak'])
```

```
0       0.0
1       1.0
2       0.0
3       1.5
4       0.0
       ...
1185    1.2
1186    3.4
1187    1.2
1188    0.0
1189    0.0
Name: oldpeak, Length: 1190, dtype: float64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['ST slope'])
```

```
0       1
1       2
2       1
3       2
```

```
4       1
       ..
1185    2
1186    2
1187    2
1188    2
1189    1
Name: ST slope, Length: 1190, dtype: int64
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
print(df['target'])
```

```
0       0
1       1
2       0
3       1
4       0
       ..
1185    1
1186    1
1187    1
1188    1
1189    0
Name: target, Length: 1190, dtype: int64
```

```
df.rename(columns={'target':'heart disease'},inplace=True)
```

```
df.columns
```

```
Index(['age', 'sex', 'chest pain type', 'resting bp s', 'cholesterol',
       'fasting blood sugar', 'resting ecg', 'max heart rate',
       'exercise angina', 'oldpeak', 'ST slope', 'heart disease'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1190 entries, 0 to 1189
Data columns (total 12 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   age                  1190 non-null   int64
 1   sex                  1190 non-null   int64
 2   chest pain type      1190 non-null   int64
 3   resting bp s         1190 non-null   int64
 4   cholesterol          1190 non-null   int64
 5   fasting blood sugar  1190 non-null   int64
 6   resting ecg          1190 non-null   int64
 7   max heart rate       1190 non-null   int64
 8   exercise angina      1190 non-null   int64
 9   oldpeak              1190 non-null   float64
 10  ST slope             1190 non-null   int64
 11  heart disease        1190 non-null   int64
dtypes: float64(1), int64(11)
memory usage: 111.7 KB
```

```
import pandas as pd
df=pd.read_csv("/content/heart disease.csv")
df.rename(columns={'target':'heart disease'},inplace=True)
df['heart disease'].value_counts()
```

heart disease	count
1	629
0	561

dtype: int64

```
df['heart disease'].value_counts(normalize=True)
```

heart disease	proportion
1	0.528571
0	0.471429

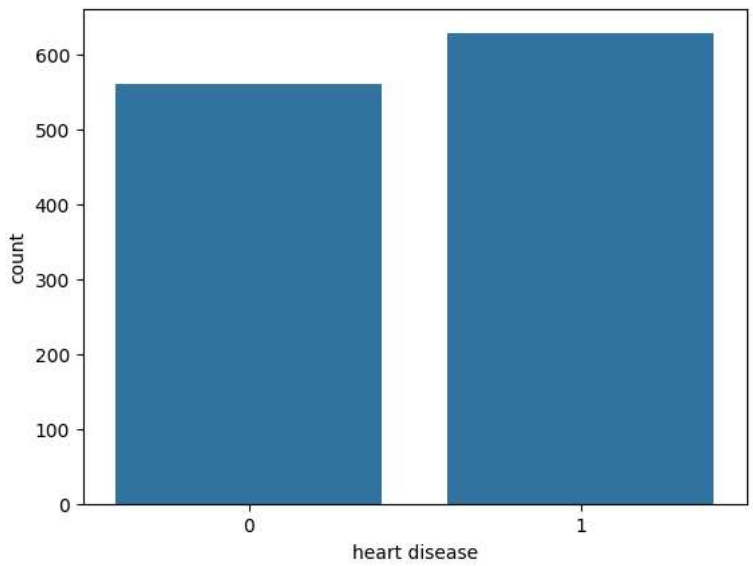
dtype: float64

```
df['heart disease'].value_counts(normalize=True)*100
```

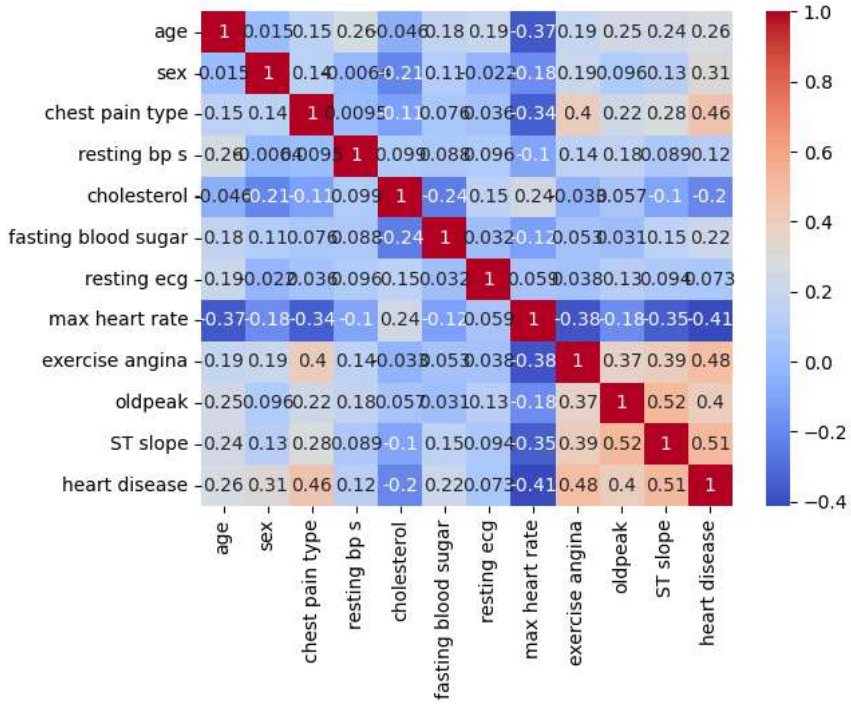
heart disease	proportion
1	52.857143
0	47.142857

dtype: float64

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x='heart disease',data=df)
plt.title=("Heart Disease Distribution")
plt.label=("heart disease","count")
plt.show()
```




```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title=("Correlation Heatmap")
plt.show()
```




```
X=df.drop('heart disease',axis=1)
y=df['heart disease']
print(X, y)
```

```
      age  sex  chest pain type  resting bp s  cholesterol  \
0      40    1                2           140          289
1      49    0                3           160          180
2      37    1                2           130          283
3      48    0                4           138          214
4      54    1                3           150          195
...   ...  ...              ...           ...          ...
1185   45    1                1           110          264
1186   68    1                4           144          193
1187   57    1                4           130          131
1188   57    0                2           130          236
1189   38    1                3           138          175

      fasting blood sugar  resting ecg  max heart rate  exercise angina  \
0                       0            0             172                0
1                       0            0             156                0
2                       0            1              98                0
3                       0            0             108                1
4                       0            0             122                0
...                   ...          ...             ...              ...
1185                    0            0             132                0
1186                    1            0             141                0
1187                    0            0             115                1
1188                    0            2             174                0
1189                    0            0             173                0

      oldpeak  ST slope
0         0.0         1
1         1.0         2
2         0.0         1
3         1.5         2
4         0.0         1
...       ...       ...
1185      1.2         2
1186      3.4         2
1187      1.2         2
1188      0.0         2
1189      0.0         1

[1190 rows x 11 columns] 0       0
1       1
2       0
3       1
4       0
       ..
1185    1
1186    1
```

```
1187    1
1188    1
1189    0
Name: heart disease, Length: 1190, dtype: int64
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X, y, test_size=0.2, random_state=42)
print("Splitting the data")
```

```
Splitting the data
```

```
import warnings
warnings.filterwarnings('ignore')
print("warning ingored")
```

```
warning ingored
```

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_train, y_train)
print("Model Trained")
print(lr.score(X_train,y_train))
```

```
Model Trained
0.8098739495798319
```

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
print("Model Trained")
print(dt.score(X_train,y_train))
```

```
Model Trained
1.0
```

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
print("Model Trained")
print(rf.score(X_train,y_train))
```

```
Model Trained
1.0
```

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print("Model Trained")
print(knn.score(X_train,y_train))
```

```
Model Trained
0.7878151260504201
```

```
y_pred_lr = lr.predict(X_test)
print(y_pred_lr)
print(len(y_pred_lr))
```

```
[1 1 0 1 0 1 0 0 1 0 0 0 0 1 1 1 1 0 0 1 1 1 1 1 1 1 0 1 0 1 1 1 0 1 1 0 1
 1 0 1 0 0 1 0 1 1 1 1 1 1 1 0 1 0 1 1 0 0 0 1 1 0 0 1 1 1 0 1 1 1 0 0 1 1
 1 0 1 1 1 1 0 0 0 1 0 0 0 0 1 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 1 0 0 0 1 1 0
 1 0 1 1 1 1 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 1 1 0 0 1 0 1 1 0 1 0 1 1 1 1
 1 1 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 0 0 0 0 1 1 1 1 0 1 1 1 0 0 1 0 1 0 0 1
 1 0 0 1 0 0 1 1 1 0 1 1 0 1 1 0 1 1 0 1 0 0 0 1 0 1 0 1 0 0 1 0 1 1 1 1 1
 1 0 0 0 1 1 1 1 0 1 1 1 0 0 1 1]
238
```

```
y_pred_dt = dt.predict(X_test)
print(y_pred_dt)
print(len(y_pred_dt))
```

```
[1 0 0 1 1 1 0 0 0 0 0 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 0 1
 1 1 0 0 0 0 0 1 1 1 1 1 1 1 0 1 0 1 1 0 0 0 1 1 0 0 0 0 1 0 1 1 1 0 0 1 1
 1 0 0 1 1 1 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 1 0 0 1 1 1 1 0 0 0 1 1 0
 1 0 1 0 1 1 0 1 0 1 1 0 0 0 0 1 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0 1 1 1 1 0 1
```

```
 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 0 1 0 1 1 1 1 1 0 1
 1 0 0 1 0 0 1 1 1 0 1 0 1 1 0 1 1 1 1 0 0 0 0 0 1 0 0 1 0 0 0 1 1 1 0 1 1
 1 0 0 1 0 1 0 1 0 1 1 1 1 0 1 1]
238
```

```
y_pred_rf = rf.predict(X_test)
print(y_pred_rf)
print(len(y_pred_rf))
```

```
[1 1 0 1 1 1 0 0 0 0 0 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 0 1
 1 1 1 0 0 1 0 1 1 1 1 1 1 1 0 1 0 1 1 0 0 0 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1
 1 0 0 1 1 1 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 1 1 0 1 1 1 1 0 0 0 1 1 0
 1 0 1 1 1 1 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1 1 1 1 1
 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0 1 1 1 1 1 1 0 1 1 1 1 1 0 1
 1 0 0 1 0 0 1 1 1 0 1 0 1 1 0 0 1 1 1 0 0 0 0 1 1 0 0 1 0 0 0 1 1 1 1 1 1
 1 0 0 1 0 1 1 1 0 1 1 1 1 0 1 1]
238
```

```
y_pred_knn = knn.predict(X_test)
print(y_pred_knn)
print(len(y_pred_knn))
```

```
[1 0 1 0 1 1 0 0 0 0 1 0 1 1 0 0 0 1 1 1 1 0 0 1 0 1 0 1 0 1 1 1 1 1 0 1 1
 1 0 1 0 0 1 0 1 1 1 0 1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 1 1 1 0 0 1 0 0 0 1 1
 1 0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 1 1 1 0 1 0 1 1 1 0 1 1 1 0 1 0 0 1 1 0
 1 1 0 1 0 1 1 1 0 1 1 0 0 0 1 1 1 1 1 1 0 0 0 1 0 0 0 0 1 1 0 1 0 0 1 0 1
 1 1 0 0 0 0 1 0 1 1 1 1 0 0 1 0 1 0 1 0 0 1 1 1 1 1 1 1 0 0 0 1 0 1 0 0 1
 1 0 0 1 0 0 0 0 1 0 1 0 0 1 0 0 1 1 1 0 1 0 0 1 1 0 0 1 1 0 0 0 1 1 1 1 0
 1 0 0 1 0 1 1 1 0 0 1 1 1 0 0 1]
238
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_lr)
print(cm)
```

```
[[ 86  21]
 [ 19 112]]
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_dt)
print(cm)
```

```
[[ 99   8]
 [ 17 114]]
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_rf)
print(cm)
```

```
[[100   7]
 [  5 126]]
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_knn)
print(cm)
```

```
[[76 31]
 [36 95]]
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred_lr)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8319327731092437
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred_dt)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8949579831932774
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred_rf)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9495798319327731
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred_knn)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.7184873949579832
```

```
from sklearn.metrics import roc_auc_score, accuracy_score, f1_score

roc_auc = roc_auc_score(y_test, y_pred_rf)
accuracy = accuracy_score(y_test, y_pred_rf)
f1_score = f1_score(y_test, y_pred_rf)
```

```
print("ROC AUC Score:", roc_auc)
print("Accuracy Score:", accuracy)
print("F1 Score:", f1_score)
```

```
ROC AUC Score: 0.9482057501605193
Accuracy Score: 0.9495798319327731
F1 Score: 0.9545454545454546
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support
```